SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING **D. Harsha Vardhan Chowdary**
COURSE CODE: CSA4303 **192372030**

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: **Total Marks: 30**

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly. **Questions:**

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body> </html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I Dabbara. Harsha Vardhan Chowdary (192372030) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weig htage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconception s	Poor understandin g; major misconceptio ns
Explanation	25%	Detailed, wellexplained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Question 1: Online Symposium Portal

1. Identify Missing HTML5 Semantic Elements

- **Form Controls:** Form inputs lack <label> elements, which are required for usability, explicit form control binding, and screen reader accessibility.

- **Document Structure:** The form exists in isolation without structural container elements such as <main>, <section>, or <article>.

- **Field Grouping:** Input fields are not visually or semantically grouped using <fieldset> and <legend>.

- **Validation Attributes:** HTML5 constraint validation attributes (e.g., required, autocomplete, placeholder) are absent.

2. Modified Standards-Compliant HTML Structure

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Online Symposium Registration</title>
</head>
<body>
  <main>
    <section>
      <h2>Symposium Registration</h2>
      <form action="/register" method="POST">
        <fieldset>
          <legend>Student Registration Details</legend>

          <p>
            <label for="student-name">Full Name:</label>
            <input type="text" id="student-name" name="name" required placeholder="John Doe">
          </p>

          <p>
            <label for="student-email">Email Address:</label>
            <input type="email" id="student-email" name="email" required
placeholder="student@example.com">
          </p>

          <button type="submit">Submit Registration</button>
        </fieldset>
      </form>
    </section>
```

```
  </main>
</body>
</html>
```

3. HTTP Request Generation

When submitted, the browser packages the inputs into an HTTP POST request payload.

HTTP Request Payload:

```
HTTP
POST /register HTTP/1.1
Host: www.example.edu
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Content-Type: application/x-www-form-urlencoded
Content-Length: 37

name=Harsha+Vardhan&email=student%40simats.edu
```

- **Method:** POST places payload data inside the HTTP request body rather than visible URL parameters.

- **Content-Type:** Defaults to application/x-www-form-urlencoded, encoding special characters (e.g., @ becomes %40, spaces become +).

4. HTTP Response Cycle Analysis

1. **Client Submission:** The browser sends the POST request over TCP/IP to port 80 (HTTP) or 443 (HTTPS).

2. **Server Processing:** The web application server parses the payload at /register, validates input, and updates database records.

3. **Server Response:** The server sends back an HTTP status code along with body content:

    - 200 OK or 201 Created: Returns a success confirmation page.

    - 303 See Other: Redirects the browser to a confirmation dashboard (Post/Redirect/Get pattern).

4. **Browser Rendering:** The browser receives headers and body payload, renders the UI update, and completes the transaction cycle.

5. Usability and Accessibility Improvements

- **Explicit** <label> **Elements:** Binds text labels directly to inputs via for and id attributes for screen readers.

- **Form Validation:** Uses built-in HTML5 attributes (required, type="email") for immediate client-side validation before server dispatch.

- **Focus Management & UX:** Includes placeholder attributes for visual context and clear button type declarations (type="submit").

Question 2: Cross-Browser Compatibility Issue

1. Identify Improper Nesting and Missing Closing Tags

- **Missing** <!DOCTYPE html>**:** Forces browsers into "Quirks Mode" rather than standard rendering mode.

- **Improper Nesting:** The tag is opened directly inside <h1> without closing <h1> first (<h1>Welcome About us). Under HTML specifications, heading tags cannot contain block-level element children like .

- **Missing Closing Tags:** Both <h1> and lack explicit closing tags (</h1>,).

2. Browser Interpretation Analysis

- **DOM Tree Construction Errors:** Modern browser parsers (HTML5 parsing specification) automatically attempt error recovery on broken markup, but do so unpredictably across engines (Blink/Gecko/WebKit).

- **Auto-Correction Behavior:**

    - Browsers auto-close <h1> immediately before encountering because block elements cannot sit inside heading nodes.

    - The resulting DOM structure becomes siblings rather than nested elements:

HTML
<h1>Welcome</h1>
About us

- **Layout Inconsistencies:** Older browsers or non-standard engines may render inside heading elements, inheriting bloated font sizes, inconsistent line heights, and broken margin/padding layouts.

3. Missing Semantic Elements

- **Missing Structural Elements:** <header>, <nav>, <main>, <section>, and <footer> are omitted.

- **Generic Containers:** Raw tags are used in place of meaningful structural landmarks.

4. Corrected W3C-Compliant HTML Structure

HTML
<!DOCTYPE **html**>
<html lang="en">

```
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Company</title>
</head>
<body>
 <header>
  <h1>Welcome</h1>
 </header>

 <main>
  <section>
   <h2>About Us</h2>
   <p>Company information and background details go here.</p>
  </section>
 </main>
</body>
</html>
```

5. Cross-Browser Compatibility Techniques

1. **W3C Validation:** Run HTML through the official W3C Markup Validation Service to fix parsing syntax errors.

2. **CSS Resets / Normalize.css:** Apply standard baseline stylesheets (normalize.css) to eliminate default browser agent styling discrepancies.

3. **Feature Detection:** Use tools like Modernizr or CSS @supports rules to handle browser feature support dynamically.

4. **Automated Cross-Browser Testing:** Test layout rendering across multiple browser engines using automated tools (BrowserStack, Playwright, Selenium).

Question 3: Form Submission Failure

1. Identify Issues in Form Configuration

- **Button Type Misconfiguration:** <button type="button">Login</button> defines a static button that triggers no default action. Clicking it does not trigger form submission.

- **Insecure HTTP Method for Credentials:** method="GET" appends sensitive form input directly onto URL query strings.

- **Lack of Validation Controls:** Input fields lack required attributes and length/pattern constraints.

2. Analysis of Why the HTTP Request Is Not Triggered

- Default browser behavior for <button> elements inside a <form> is type="submit".

- However, setting type="button" explicitly overrides this default, converting it into a generic button designed only for client-side JavaScript click handlers.

- Because no JavaScript listener or submit action is attached, clicking the button performs no action, preventing any HTTP request from firing.

3. Role of GET Method in This Scenario

- **Data Exposure:** GET appends parameters directly to the URL string (e.g., /submit?username=admin&password=12345).

- **Security Risk:** Sensitive authentication data gets saved in plaintext across browser history logs, web server access logs, network proxy caches, and referrer headers.

- **Idempotency Misuse:** GET requests are intended for safe, repeatable data fetches without side effects—never for state-changing or sensitive actions like account login.

4. Corrected Implementation

HTML

```
<form action="/submit" method="POST">
 <p>
  <label for="username">Username:</label>
  <input type="text" id="username" name="username" required autocomplete="username">
 </p>

 <p>
  <label for="password">Password:</label>
  <input type="password" id="password" name="password" required autocomplete="current-password">
 </p>

 <button type="submit">Login</button>
</form>
```

5. Improvements for Secure Data Transmission

- **Enforce HTTPS (TLS/SSL):** Encrypt all communication channels so form data in transit cannot be read or tampered with by third parties.

- **Use POST Method:** Transmit authentication credentials inside the HTTP request body instead of visible URL strings.

- **Implement CSRF Protection:** Inject unique, cryptographically secure anti-CSRF tokens into every form submission to block cross-site request forgery attacks.

- **Secure Input & Headers:** Enforce HTTPS redirects via HSTS headers and apply strict client/server input sanitation.